



FIREBRAND

LYCEUM EDUCATION GROUP

Python Fundamentals

Python Pathway | Session 1

Session Recording & Professional Expectations

Session Recording

- All Estio training sessions are recorded to:
 - Support learner progress
 - Enable review of content
 - Maintain quality and compliance standards
- By attending this session, you acknowledge that recording is taking place.

Camera & Participation Expectations

- To support engagement and attendance monitoring:
 - Cameras should remain on during the session
 - Please participate actively in discussions and activities
 - If you are unable to use your camera or microphone for any reason, please inform your tutor directly via chat or message.

This session forms part of your professional apprenticeship experience. Please engage respectfully and professionally throughout.

Safeguarding, British Values & Wider Learning

Safeguarding protects individuals from harm, abuse, neglect, radicalisation and exploitation.

If you have a concern about yourself or someone else, email:

safeguarding@bpp.com

Or speak to your tutor privately.

We treat all concerns seriously and confidentially.

Safeguarding



The British Values are:

1. **Democracy**
2. **Rule of Law**
3. **Individual Liberty**
4. **Mutual Respect**
5. **Tolerance of Different Faiths and Beliefs**

These values underpin everything we do in professional and digital environments.

British Values



Prevent is about protecting people from being drawn into extremism or radicalisation.

In our programmes, Prevent links to:

- **Online safety**
- **Digital responsibility**
- **Critical thinking**
- **Professional ethics**

Prevent



As we go through today's session, consider:

How today's topic connects to British Values

Whether there are any Prevent or digital safety themes

How this learning supports your professional and career development

Your tutor will guide discussion and help you make these connections.

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, K7, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, **K9**, K11

Skills

S1, S4, S6, **S7**, S10,
S11, S12, S16

Behaviours

B2, B3

Lesson Aim and Objectives

Lesson Aim: Python Fundamentals: Syntax, Variables & Program Structure

Lesson Objectives

Apply

- Apply Python syntax rules to construct simple executable programs

Demonstrate

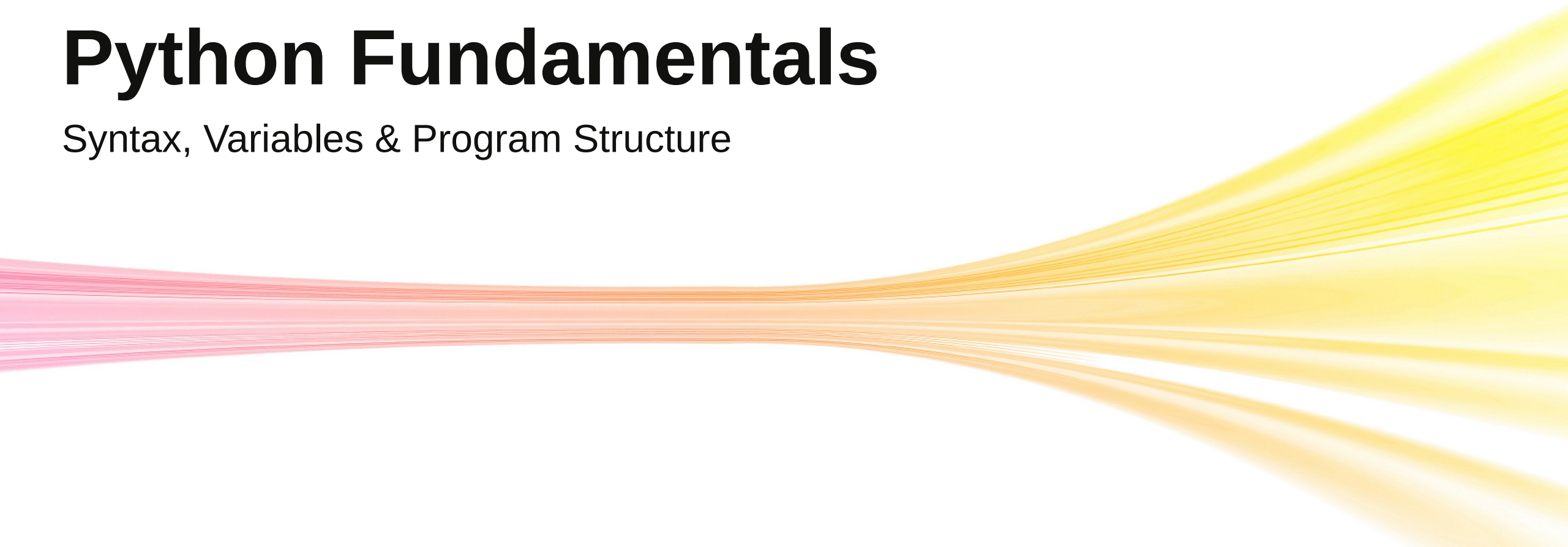
- Demonstrate use of variables and data types within a structured script

Analyse

- Analyse program flow and identify syntax errors affecting execution

Python Fundamentals

Syntax, Variables & Program Structure



What is Python?



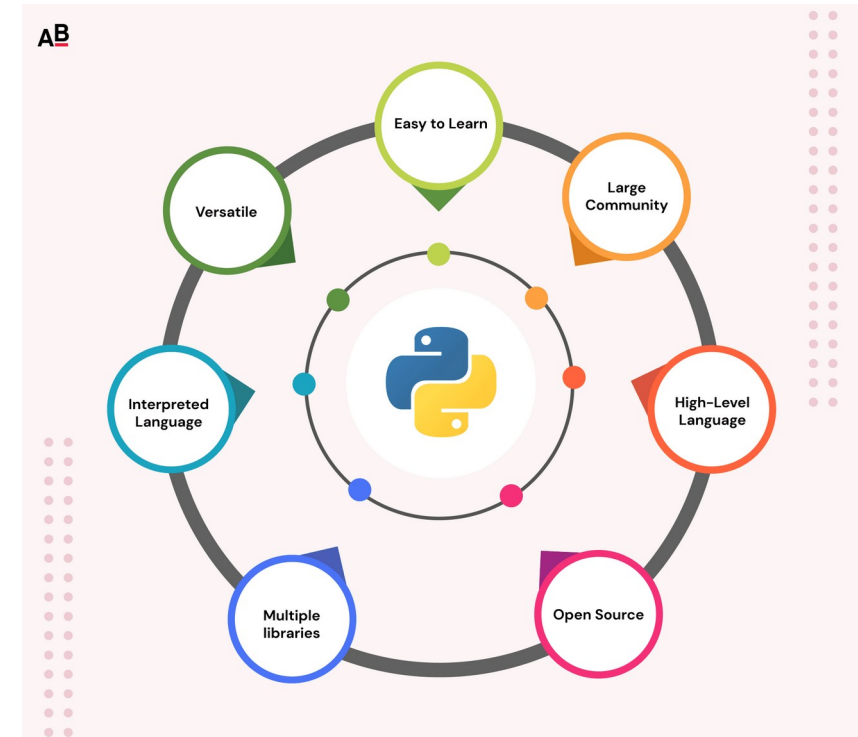
Welcome to Python!

How Python Executes Code

- Python is an interpreted language that runs code from top to bottom
- It works on a 'Line-by-Line Execution' basis

Example:

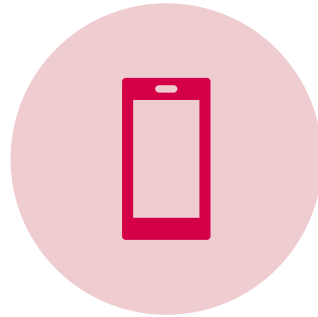
```
print("Step 1: Preparing ingredients.")  
print("Step 2: Cooking the meal.")  
print("Step 3: Dinner is served!")
```



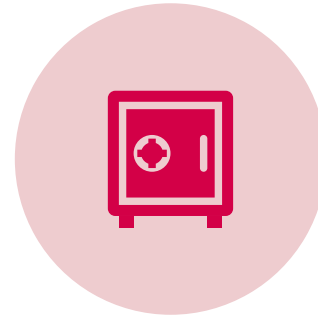
Core Characteristics



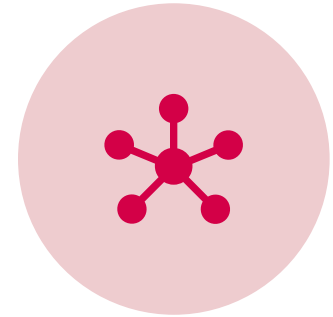
INTERPRETED



**DYNAMICALLY
TYPED**



**CROSS-
PLATFORM**



**EXTENSIVE
ECOSYSTEM**

Core Syntax Rules

Key Rules:

Rule	Description
Indentation	Spaces define blocks of code (no curly braces {} needed)
Structure	Colons : introduce blocks like loops, functions, or conditionals
Brackets	Parentheses () are used for calling functions like print()
Quotation Marks	Strings must be wrapped in matching single " or double "" quotes

Consistency is key; mixing spaces and tabs will crash your script

Right vs. Wrong (Syntax Comparison)



Correct Script:

```
if True:  
    print("This is correct!")
```

Flawed Script:

```
if True  
print("This is incorrect!")
```

What are the 3 errors in the flawed version?

- missing colon,
- missing indentation,
- an unclosed parenthesis

Corrected Script:

```
if True:  
    print("This is now correct!")
```

Understanding Error Messages

Error messages are helpful clues, not failures

Common Error Breakdown:

- **SyntaxError: expected ':'** → You forgot a colon at the end of a line
- **IndentationError: unexpected indent** → Your code alignment is incorrect
- **NameError: name 'prnt' is not defined** → You made a typo in a function name

Tip: Always look at the last line of the error block first to find the exact issue

Anatomy of a Python Error Message

Deconstructing a Traceback:

```
File "script.py", line 3
  print("Hello"
      ^
SyntaxError: incomplete input
```

Key Components:

- **The File & Line:** Tells you exactly where Python got confused (script.py, line 3)
- **The Visual Indicator:** The caret symbol (^) points to where the error was detected
- **The Error Name:** Categorises the issue (e.g., SyntaxError, TypeError, ValueError)
- **The Description:** Provides the specific detail (incomplete input means a closing bracket or quote is missing)

Debugging Rule: If line 3 looks perfectly fine, check line 2; errors sometimes bleed over from the uncompleted line above

Variables and Data Types

- Variables are for **Storing** and **Updating** data, they're like containers that hold different kinds of information

Key Data Types:

- **Integer (int)**: Whole numbers, like `score = 25`
- **Float (float)**: Decimal numbers, like `price = 19.99`
- **String (str)**: Text data, like `city = "London"`
- **Boolean (bool)**: True/False values, like `is_active = True`

Updating Value: You can change a variable's data later in the script (e.g., `score = score + 5`)

^ What does this script do?

Constants

- A **constant** is a type of variable whose value **cannot be changed** once it is set during the running of a program

Why Use Constants?

- **Prevents Accidents:** It stops you or other programmers from accidentally overwriting critical data (like changing the value of Pi by mistake)
- **Easy Maintenance:** If a global value changes in the real world (like a VAT rate), you only have to update it in **one place** at the top of your code
- **Code Clarity:** Seeing `TOTAL * VAT_RATE` is much easier to understand than seeing `TOTAL * 0.20`

Formatted Output with print()

Mastering the print() Statement

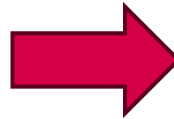
- Print() is used to display clear, formatted data to the user

Key Techniques:

- **Basic Printing:** Passing text or numbers into print()
- **F-Strings:** Embedding variables directly inside strings using f"Text {variable}"

Code Example:

```
name = "Alex"  
score = 95  
print(f"User {name} scored {score} points!")
```



User Alex scored 95 points!

Interactive Scripts with input()

- We can use input() to **Capture** and **Process** user input
- It makes programs interactive by taking text from the keyboard

The Golden Rule: The input() function ***always*** returns a **string** data type

Type Conversion: We can convert that string into a number so you can do math

Code Example:

```
age_str = input("Enter your age: ")  
age = int(age_str) # Converts string to integer  
print(f"Next year you will be {age + 1}")
```

Next year you will be 26

What will the output of this script look like if the user entered the number 25?

Benefits of String Formatting

1. Superior Readability

- Keeps your text intact as a single sentence. With older methods like concatenation (+), your code gets chopped up into multiple fragments, making it incredibly hard to read at a glance

2. Automatic Data Type Conversion

- Automatically converts numbers, Booleans, and lists into readable text for you behind the scenes

3. Inline Code Execution

- Allows Python expressions to run inside the text. You can perform math, call functions, or modify text capitalisation directly inside the placeholders

4. Built-in Text & Number Styling

- You can format numbers without altering the original data variables, allowing you to:
 - Round long decimals (e.g., limiting 3.14159 to just 3.14)
 - Format numbers as currency (e.g., adding commas to 1000000 to show 1,000,000)
 - Pad numbers with leading zeros (e.g., turning 5 into 05 for clock displays)
 - Align text into neat columns for terminal tables

The Code-Run-Modify Cycle

- We use this method when **Experimenting** and **Observing** the behaviour of our script
- Developing code is an iterative process of trial and observation

The Workflow:

- **Write:** Add or change a line of code in your script
- **Run:** Execute the program to see the machine's response
- **Observe:** Look at how the output or errors change

Best Practice: Change only one thing at a time so you know exactly what caused a new behaviour



FIREBRAND

LYCEUM EDUCATION GROUP

Demo

Python_Demo_1.py

Demo step-by-step

- Open **Python_Demo_1.py** from the Git Repo (it is recommended to use Visual Studio Code)
- Run the code to make sure it functions properly

Task 1:

- Try changing the data types (e.g., change `item_price` to a whole number)
- Break the code intentionally (e.g., delete a colon or a parenthesis) to practice reading the resulting error message

Task 2: Add VAT

- Update the total price to include 20% VAT

Task 3: Add a Discount

- Add a flag (Boolean) to determine whether a discount should be applied
- Deduct a flat £2.00 off the order total

Daily Work Logger



ACTIVITY



Objective: Build a simple time-tracking command-line application

Core Core Requirements:

- Capture user text inputs
- Store data in variables
- Display formatted summaries

Key Challenge: Ensure data types match your tracking needs

Extension Activity: Capture multiple entries and calculate total duration



ACTIVITY



Structured Debugging

We need to shift from random trial-and-error to a methodical debugging strategy

- **Why It Matters:**

Guessing changes wastes time; reading clues solves bugs permanently

- **The Mission:**

Analyse pre-written, broken code scripts and repair them systematically

- **Expected Outcome:**

Documented evidence of a logical troubleshooting approach.

Structured Debugging Activity



ACTIVITY



Open the file **Activity_1.py** from the Git Repo

Objective: Evidence a structured approach to debugging

Task 1: Identify and Annotate:

Find the bug and write a brief comment explaining the flaw

Task 2: Correct and Re-run:

Fix the specific problem line and test the file execution

Task 3: Classify the Type:

Label the mistake to understand exactly why it failed (e.g., syntax, runtime, logic)



ACTIVITY



Introduction to Code Quality

We want to evaluate scripts beyond whether they simply "work" or "run"

- **Why It Matters:**

- Code is read far more often than it is written

- Programs must be built for human developers to understand, not just for computers to run

- **The Mission:**

- Review, critique, and audit real-world code solutions

- Evaluate structural layouts, name clarity, and documentation habits

- **Expected Outcome:**

- Clearly articulate what makes an application clear and maintainable

- Write scripts that teammates can immediately understand and modify

Code Quality Activity



Open the file **Activity_2.py** from the Git Repo

Objective: Articulate what makes code clear and maintainable

Task 1: Identify and Annotate:

Find the problem areas and annotate them using comments (e.g., using #)

Task 2: Correct and Re-run:

Re-write the code to make it more readable

Task 3: Classify the Type:

Label the mistake to understand exactly why it failed (e.g., syntax, runtime, logic)



ACTIVITY



Stretch and Challenge

The Mission:

Upgrade your working logger application into production-grade code

- **Task 1: Consistent Naming Conventions**

Convert every variable name to standard Python snake_case

Ensure zero single-letter variables remain in your script

- **Task 2: Structural Documentation**

Add meaningful comments explaining the underlying logic.

Use visual dividers to separate input, processing, and output sections.

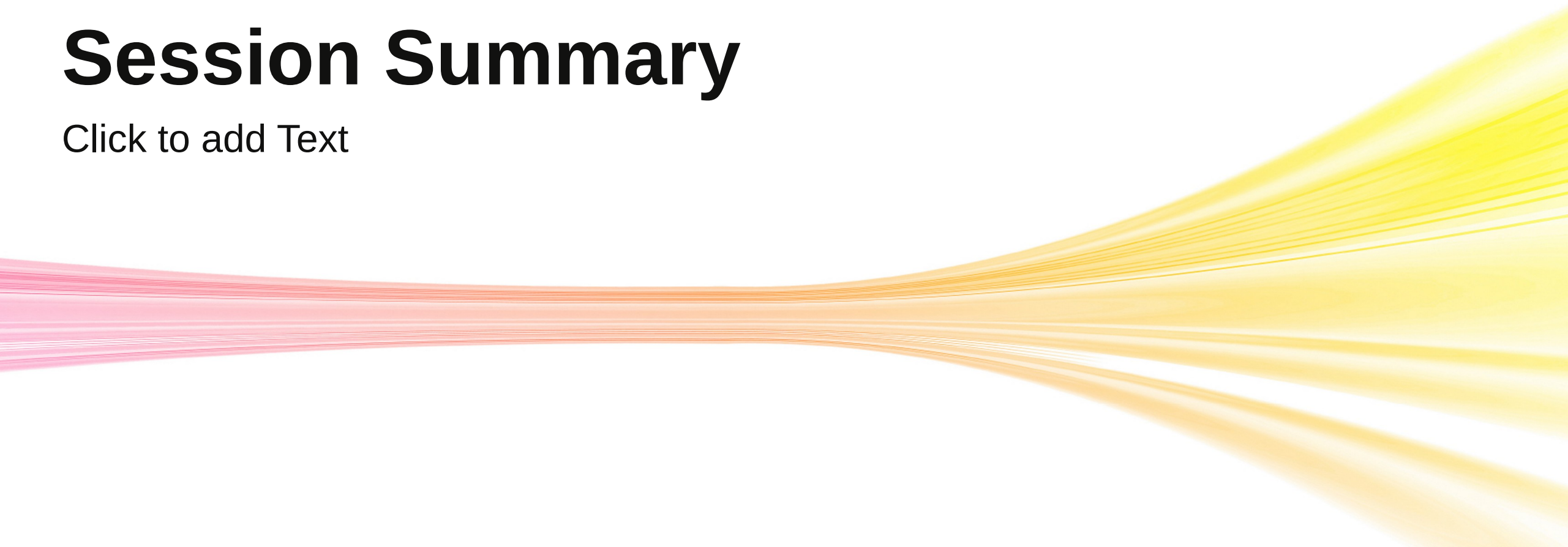
- **Task 3: Premium Output Formatting**

Redesign the terminal receipt using aligned spacing and structural borders.

Make the data instantly scannable for a manager or client review.

Session Summary

Click to add Text



Lesson Aim and Objectives

Lesson Aim: Python Fundamentals: Syntax, Variables & Program Structure

Lesson Objectives

Apply

- Apply Python syntax rules to construct simple executable programs

Demonstrate

- Demonstrate use of variables and data types within a structured script

Analyse

- Analyse program flow and identify syntax errors affecting execution

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, K7, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, **K9**, K11

Skills

S1, S4, S6, **S7**, S10,
S11, S12, S16

Behaviours

B2, B3

